Text Analysis with Enhanced Annotated Suffix Trees: Algorithms and Implementation

Mikhail Dubov(✉)

Computer Science Faculty, National Research University Higher School of Economics, Moscow, Russia
mdubov@hse.ru

Abstract. We present an improved implementation of the Annotated suffix tree method for text analysis (abbreviated as the AST-method). Annotated suffix trees are an extension of the original suffix tree data structure, with nodes labeled by occurrence frequencies for corresponding substrings in the input text collection. They have a range of interesting applications in text analysis, such as language-independent computation of a matching score for a keyphrase against some text collection. In our enhanced implementation, new algorithms and data structures (suffix arrays used instead of the traditional but heavyweight suffix trees) have enabled us to derive an implementation superior to the previous ones in terms of both memory consumption (10 times less memory) and runtime. We describe an open-source statistical text analysis software package, called "EAST", which implements this enhanced annotated suffix tree method. Besides, the EAST package includes an adaptation of a distributional synonym extraction algorithm that supports the Russian language and allows us to achieve better results in keyphrase matching.

Keywords: Text analysis · Algorithms on strings · Annotated suffix trees · Suffix arrays · Synonym extraction

1 Introduction

The annotated suffix tree (AST) method has first been introduced in 2006 by Pampapathi, Mirkin, and Levene [12] as a novel text classification tool with application to anti-spam e-mail filtering. It differs from the majority of text processing methodologies in that it considers the text as a sequence of letters, not as a set of words. While the latter approach, called the *bag-of-words* model, is much more commonly exploited both in literature and in practice (e.g. [13, pp. 167–200]), the former one has the advantage that it makes the text analysis more language-independent [10]. Indeed, with a letter-based approach there is usually no need to perform such preprocessing step as stemming since the performance of algorithms no longer depends on the uniformity of grammatical word forms. The annotated suffix tree method allows omitting the stop words collection and the filtering step as well since it is not sensitive to the "noise" in the text. Seemingly, the most significant feature of this approach is that it captures the sequential structure of the texts being analyzed [11].

M.Y. Khachay et al. (Eds.): AIST 2015, CCIS 542, pp. 308–319, 2015.
DOI: 10.1007/978-3-319-26123-2_30

The basic computation in the AST method is the matching (relevance) score for some keyphrase against the input text collection indexed by the AST. This "approximate" matching score, lying in the $[0; 1]$ interval, serves as a basis for several interesting applications. Among the most interesting examples in this field, let us mention the AST-based feature extraction and text classification algorithms presented by Pampapathi [12], and also the work by Mirkin, Chernyak, & Chugunova [10], where the AST method is exploited to build a keyphrase reference graph.

While the annotated suffix tree methodology is a very promising tool for a great variety of text analysis applications, it has proven to be not efficient enough in terms of running time and space consumption to be able to process large-scale text collections. In this paper, we describe an enhanced implementation of this method. This implementation uses suffix arrays instead of pure suffix trees as the main underlying data structure, which allows it to require up to 10 times less memory and be faster than any other previous implementation of the AST method known to us. We also discuss injecting one language-dependent feature into the AST method, namely the context-based extraction of synonyms, which is neccessary for more accurate keyphrase relevance score computation.

We structure this paper as follows: in Sect. 2, we describe the foundations of the original AST method. In Sect. 3, we review the suffix array data structure and show how it can substitute suffix trees in an AST method implementation. We devote Sect. 4 to a Python package called EAST that implements these algorithms and describe some of its features. Finally, we discuss some experimental results of our study obtained using this package in Sect. 5.

2 Annotated Suffix Trees

A *suffix tree* is considered to be one of the central data structures in the field of string processing algorithms [5, pp. 89–93]. Its importance stems from the fact that it establishes a linear time solution for one of the classical tasks in strings processing, namely the *exact pattern matching* problem, and serves as an efficient data structure for *full-text indexing*. Formally, the suffix tree data structure for a string S ($|S| = n$) is defined as a rooted directed tree encoding all the suffixes of that string. It is constructed in such a way that the concatenation of edge labels on every path from the root node to one of the leaves makes up one of the suffixes of that string, i.e. $S[i \ldots n]$. It is also required that each internal node has two or more children, and each edge is labeled with a non-empty substring of S [5, p. 90]. Suffix trees can be constructed in linear time and stored in linear space [15]. A generalized suffix tree is a suffix tree built for multiple strings; it can be constructed in linear time as well [5, p. 116].

While being linear in size and in construction runtime, suffix trees still suffer from extensive memory consumption: a typical implementation needs 30 bytes of memory on average per one input symbol. Besides, suffix trees have poor locality of memory reference, which leads to a loss of efficiency on modern cached processor architectures [1].

An *annotated suffix tree* is an extension of the original suffix tree data structure. In addition to the edge labels in suffix trees, annotated suffix trees have their nodes labeled with integer numbers indicating the number of entries of the substring on the path from the root to that node (as spelled out by the edge labels) in the original text collection. These node annotations can then be used to calculate the matching score (a real number in [0; 1]) of a given keyphrase against the text the annotated suffix tree has been built for [10]. In this way, annotated suffix trees provide an alternative solution to the *approximate pattern matching* problem.

For example, when built for a single string *"XABXAC"*, the annotated suffix tree gives us a relevance score of 0.25 for the string *"ABC"* and 0.11 for the string *"XYZ"* (which has only one common letter with *"XABXAC"*). When used to index the text of *"Alice in Wonderland"* by L. Carroll[1], the AST returns 0.32 as a matching score for *"Alice"* and 0.04 for *"Bob"* (as expected). This score can be thus seen as a measure of relevance of a keyphrase to some text collection (in our experience, values greater than 0.2 are usually strong indicators of relevance).

In the first publications [12], the annotated suffix tree has been naively depicted as a tree where each node v contains exactly one letter and stores the value of $f(v)$, which is the number of occurrences of a substring, obtained by the concatenation of all the symbols on the path from root to that node, in the original text collection (see Fig. 1). In this representation, the tree is actually not a suffix but a *keyword tree* (a *trie*), which requires quadratic space to store and therefore quadratic time to be constructed. It allows, however, to make a definition of the key phase matching score that is easy to state and to interpret.

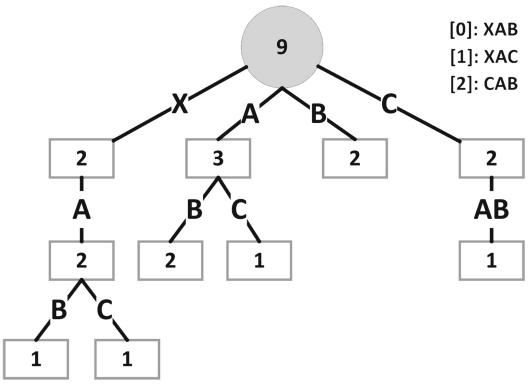

Fig. 1. Naive AST representation

To describe the matching score computation procedure for the naive AST representation, let us define the *conditional probability of a node* given its parent as

$$\hat{p}(v) = \frac{f(v)}{f(parent(v))}$$

[1] http://archive.org/stream/alicesadventures19033gut/19033.txt.

Where v is a node, $f(v)$ is the node's annotation (the frequency of the corresponding substring) and $parent(v)$ is the parent node of v. Imagine that we try to match some sequence of letters against the suffix tree just as in the exact pattern matching problem, starting from the root and proceeding to the child nodes. If, for some string s, we matched exactly k symbols in the tree, then

$$score_{seq}(s) = \frac{\sum_{i=1}^{k} \hat{p}(v_i)}{k},$$

Where v_i is the i-th node on the matching path starting at the root. Now we are ready to define the actual AST matching score:

Definition 1. *AST matching score* for a given keyphrase S against some text collection T is a value computed for $AST(T)$ as

$$SCORE(S) = \frac{\sum_{i=1}^{|S|} score_{seq}(S[i:])}{|S|}$$

In other words, we first match each suffix of the input keyphrase against the constructed annotated suffix tree in terms of the *conditional probabilities* of the nodes corresponding to its symbols and then produce the final AST matching score by averaging these suffix scores. This final matching score has can be interpreted as the *average conditional probability* of an occurrence of a single symbol of the input keyphrase in the text collection the AST has been built for.

Obviously, quadratic runtime and memory consumption of such a naive implementation of the AST methodology as described above are unsatisfactory for real applications. Optimized implementations are based on the actual suffix tree data structure (not on a keyword tree) with compacted edges and no *"chain"* nodes, i.e. having exactly one child (Fig. 2, top-level leaves for unique symbols omitted for simplicity).

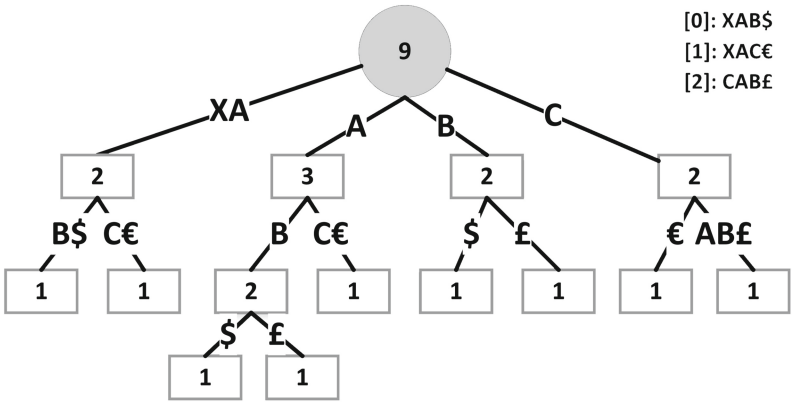

Fig. 2. Optimized AST representation

It has been shown by the author [3] that a very simple preprocessing of the text collection (appending unique termination characters to each text) makes it possible to annotate the tree with substring frequencies $after$ the pure suffix tree was constructed for the preprocessed text collection (using some linear algorithm like the *Ukkonen's algorithm*). Indeed, with unique symbols appended, the number of occurrences for each text is guaranteed to be equal to 1, so that the leaf nodes can be annotated with 1. Then, in a bottom-up tree traversal, each inner node gets annotated with the sum of the annotations of its children. This allows to obtain a $O(n)$ algorithm for AST construction [3]:

Algorithm. **LinearASTConstruction(C)**
Input. String collection $C = \{S_1, \ldots, S_m\}$
Output. Generalized annotated suffix tree for C.

1. Construct $C' = \{S_1\$_1, \ldots, S_m\$_m\}$, where $\$_i$ are unique characters that do not appear in $S_1 \ldots S_m$.
2. Construct a generalized suffix tree T for collection C' using a linear-time algorithm (e.g. the *Ukkonen algorithm*).
3. **for** l **in** $leaves(T)$
4. **do** set $f(l) \leftarrow 1$
5. Run a postfix depth-first tree traversal on the suffix tree T. For each inner node v, set $f(v) \leftarrow \sum_{u \in children(v)} f(u)$.

The compacted annotated suffix tree representation requires only a minor change in the formula for the helper score:

$$score_{seq}(s) = \frac{\sum_{i=1}^{k} \hat{p}(v_i) + l - k}{l},$$

Since for all *"chain"* nodes that had exactly one child in the naive representation and were removed, their conditional probability $\hat{p}(v) = 1$ (in the formula k is, as before, the number of nodes in the match and l is the number of symbols in the match).

These approximate matching scores for various keyphrases alone allow us to solve a number of interesting problems. For example, having an annotated suffix tree built for a spam corpus, one may split the analyzed e-mails into keyphrases and use the AST score to determine whether the e-mail text is likely to contain a spam message [12].

3 Enhanced AST Method with Suffix Arrays

Suffix arrays have first been introduced in the work by Manber and Myers [9] as a space efficient alternative to suffix trees. Technically, a *suffix array* for a string S of length n is an array of n integer numbers, enumerating the n suffixes of S in lexicographic order [5, p. 149]. Linear-time algorithms for suffix array construction are known as well [6], but tend to be more complicated than their suffix tree counterparts.

The potentiality of suffix arrays to serve as a space efficient replacement for suffix trees has naturally lead to an intense interest to the problem of effective porting of algorithms that use suffix trees as an auxiliary data structure to suffix arrays [1,7]. It is important to note that not only suffix arrays can yield a significant gain in space efficiency, but they also can speed up certain algorithms: modern processors make use of the fact that suffix arrays are a more compact data structure than suffix trees and utilize several low-level optimizations [1].

Abouelhoda, Kurtz, & Ohlebusch [1] have shown how to systematically replace every algorithm that uses suffix trees with another one based on suffix arrays by introducing the concept of an *lcp-interval tree*. The nodes of such a tree correspond to those of the original suffix tree, but the tree itself actually does not have to be stored in memory: it is sufficient to enhance the suffix array with two auxiliary arrays called *lcp-table* and *child-table*. It has been shown that it is possible to perform both bottom-up and top-down suffix tree traversals using these arrays without loss in asymptotic time complexity. Together with the original *suffix array*, these two auxiliary arrays (*lcp* and *child*) take no more than 10 bytes per symbol in the input text collection, when implemented accurately. Thus, using suffix arrays instead of suffix trees is at least twice as efficient in terms of memory usage (because of the additional arrays used to achieve this result, the actual data structure became known as an *enhanced suffix array*).

A sample enhanced suffix array for string *"XABXAC"* is presented in Table 1. The suffixes in the last column are not actually stored in memory. There are also *three child-tables* instead of one mentioned above; we keep them to be consistent with analogous illustrations in [1]. With certain implementation tricks, these three arrays can be compacted to one.

Table 1. Enhanced suffix array for string "XABXAC"

i	Suffix array	lcp-table	Child-table			S[suff[i]:]
			1.	2.	3.	
0	1	0		1	2	ABXAC
1	4	1				AC
2	2	0	1		3	BXAC
3	5	0			4	C
4	0	0				XABXAC
5	3	2				XAC

Let us briefly mention that enhanced suffix arrays are not the only way to solve the problem of extensive memory consumption by suffix trees. Another approach is to store suffix trees in external memory [2]. Besides, there are methods to compress suffix trees in the main memory [14]. These methods, however, do not yield a compact representation of the full-text index in memory and thus disable possible low-level optimizations implemented in modern processors.

In what follows, we will show how one can further enhance this data structure to simulate node annotations and use them to compute the AST matching score. Recall that the enhanced suffix array, built for a text collection of total length of n symbols, is a set of three arrays of length n. In the annotated suffix tree, however, the total number of nodes was larger than n and each node stored a substring frequency annotation.

In fact, it is easy to overcome this issue. It follows from the definition of a suffix tree that the number of internal nodes cannot exceed $(n-1)$. Recall that, to build an annotated suffix tree, we first made each text in the collection to end with a unique symbol and then annotated all the leaves with 1. But that means that we can omit storing these leaf annotations explicitly, because we always know that they are all equal to 1. We only need to store $(n-1)$ annotations for the root and internal nodes, and we can introduce yet another auxiliary *annotation-table* of length n for that purpose. Thus, this further enhanced data structure (that we call *"enhanced annotated suffix array"*) is a set of four arrays of length n: *suffix array*, *lcp-table*, *child-table*, and *annotation-table*.

Further, we need to define how exactly the frequency annotations of inner nodes of a suffix tree should be stored in this new *annotation-table* of length n. In the *virtual lcp-tree* model mentioned above, it is possible to restore the original suffix tree using the information from the *lcp-table*. The nodes of this *lcp-tree* have a one-to-one correspondence with inner nodes of a suffix tree and are represented using three parameters $\langle l, i, j \rangle$: the *lcp-value* l and the left and right boundaries of the *lcp-interval* (i, j). It can be shown (as done in [1]) that for each *lcp-interval* $v = \langle l, i, j \rangle$ there exists a unique index, $index(v) \in [0; n-1]$, which is equal to the smallest k, such that $k > i$ and $lcp[k] = l$. It is this mapping that we use to store the inner node frequency annotations.

In Table 2, there is an example of an enhanced annotated suffix array constructed for a single string "XABXAC" (the annotated suffix tree for the same string can be seen in Fig. 3, unique symbols omitted for simplicity). We need to store only 3 numbers in the *annotation-table* since the corresponding tree has only two inner nodes annotated with 2, plus the root (it's frequency annotation is 6).

Table 2. Enhanced annotated suffix array for string "XABXAC"

i	Suffix array	lcp-table	Child-table			Annotation-table	S[suff[i]:]
			1.	2.	3.		
0	1	0		1	2	6	ABXAC
1	4	1				2	AC
2	2	0	1		3		BXAC
3	5	0			4		C
4	0	0					XABXAC
5	3	2				2	XAC

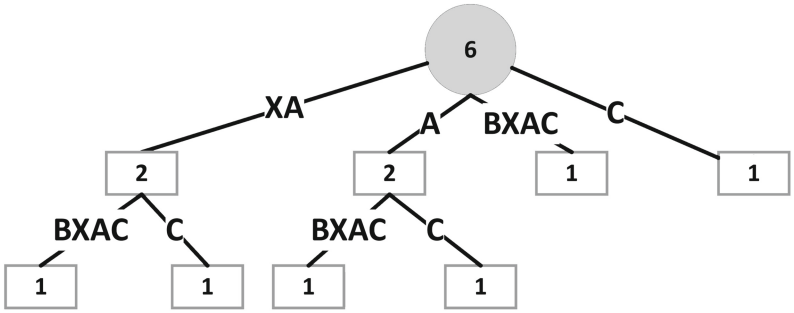

Fig. 3. Annotated suffix tree for string "XABXAC"

We are now ready to present a linear enhanced annotated suffix array construction algorithm. This algorithm has a structure which is very similar to that of **LinearASTConstruction**. First, it preprocesses the input text collection by appending unique symbols to each of its texts and builds and enhanced suffix array for the concatenation of these texts. In the last step, the algorithm simulates a bottom-up tree traversal to fill up the *annotation-table*.

Algorithm. **LinearEASAConstruction(C)**
Input. String collection $C = \{S_1, \ldots, S_m\}$
Output. Enhanced suffix array for C with substring frequency annotations.

1. Construct a string $S = S_1\$_1 + \cdots + S_m\$_m$, where $\$_i$ are unique symbols that do not appear in $S_1 \ldots S_m$ and "+" is string concatenation operator.
2. Construct a suffix array A for string S using a linear-time algorithm (e.g. the *Kärkkäinen-Sanders algorithm*) and two auxiliary arrays: *lcp-array* and *child-array*.
3. Simulate a postfix depth-first tree traversal on the suffix array A. At each of the *virtual inner nodes*, corresponding to an *lcp-interval* $v = \langle l, i, j \rangle$, where $i < j$, set $annotation[index(v)] = \sum_{u \in children(v)} annotation[index(u)] + \#(\langle l, i, j \rangle : i = j)$.

Since in the *virtual lcp-tree* model there is no processing of leaves, we have to take them into account by appending the number of *lcp-intervals* with $i = j$ at each virtual inner node in step 3. If the length of the i-th string in the collection $|S_i| = n_i$, then the runtime complexity of this algorithm is $\Theta(n_1 + \cdots + n_m)$.

The formula for the AST matching score stays unchanged with this new suffix array-based implementation: indeed, this data structure allows us to simulate not only postfix, but also prefix tree traversals [1], which is exactly what is needed for the score computation described in Sect. 2.

One last detail about the AST method implementation is that it is worth splitting every text in the input collection into a sequence of substrings, each consisting of k words, where k should be the expected number of words in the input keyphrases (usually $k = 3$ or $k = 4$). This approach has proven to provide more accurate matching scores in most practical applications [10].

4 EAST Software Package

The algorithms decribed above were implemented in a Python package EAST[2] (EAST stands for "Enhanced Annotated Suffix Trees"). This software tool is distributed as an open-source application under the MIT license. It can be retrieved from the Python Package Index system and installed via the *pip* tool.

EAST not only implements the enhanced annotated suffix arrays data structure, but also relies on the extensive usage of the *numpy* library to increase memory efficiency compared to previous implementations. This allowed us to get a significant increase in memory efficiency which we will discuss in Sect. 5. The underlying algorithms used in *EAST* are the *Kärkkäinen-Sanders algorithm* for suffix array construction [6] and the *Kasai algorithm* for lcp-table computation [7]. The package also includes the previous implementation of the AST method which uses the *Ukkonen algorithm* for suffix tree construction [15]. This implementation can be used as an alternative one.

The package not only can be used as a Python library, but also provides the user with a command line interface. For example, to produce a table of matching scores for a set of keyphrases against some text collection, one should run:

```
$ east keyphrases table <keyphrases_list.txt>
  <path/to/the/text/collection/>
```

An interesting feature of the *EAST* package is that it introduces *synonym extraction*, a language-dependent feature to enhance the AST method even more (AST is basically language-independent). Its purpose is to handle situations when one tries to match a keyphrase (say, *"plant taxonomy"*) against some text collection that may cover the topic corresponding to that keyphrase, but mostly using a different terminology (say, *"plant classification"*) and, as a result, the relevance score is unreasonably small.

The extensive body of literature concerning synonym extraction comprises two basic approaches: building synonymic thesauri based on some already existing *lexica* (dictionaries), and the analysis of word collocations in *corpora* (the so-called *distributional* approach) [16]. The former strategy usually includes the detailed analysis of the dictionary definitions structure, while the latter one is based on the assumption that similar words appear in similar contexts.

In our software, we use a distributional synonym extraction algorithm similar to that of Lin [8]. The advantage of this approach, which is of huge importance to us, is the fact that it extracts synonyms based on the same set of texts that is analyzed later using the AST method. Consequently, it is, on the one hand, capable of extracting domain-specific synonyms, and, on the other hand, does not extract redundant synonyms not present in the text collection, thus bringing no performance decrease to our algorithms.

We use the resulting set of synonyms as follows: for each input keyphrase query S against the constructed annotated suffix tree (array), we first find a set of synonymous queries $syn(S) = w_1, w_2, ..., w_k$. We then define the matching

[2] https://pypi.python.org/pypi/EAST.

score of S as $\max_{w \in syn(S)} SCORE(w)$, where *SCORE(w)* is the matching score for keyphrase w as computed by the original AST method.

The distributional synonym extraction algorithm by Lin [8] employs the so-called dependency triples (w_1, r, w_2) that model semantic relationships between words in the analyzed texts and then allow to extract words that often appear in common dependency triples as synonyms. In his original paper, Lin uses a *broad-coverage parser* to extract dependency triples from English corpora. In our adaptation of that algorithm to the Russian language, we use the *Yandex Tomita parser*[3] to extract such triples based on a set of grammatical templates like *"adjective + substantive"* or *"verb + arverb"*.

5 Experimental Results

We have conducted a series of experiments based on the implementation described in Sect. 4. To compare our new suffix array-based AST implementation against the algorithms used previously, we have measured both their runtime and memory consumption. In our experiments, we used randomly generated text collections. Each text collection contained 100 random strings, with their lengths increasing from 10 to 1000 from iteration to iteration. At each iteration, the strings in the collection started with the same prefix and differed only in several ending symbols. Such collections not only allowed us to get close to the worst-case for the naive construction algorithm, but also represented a situation that often occurrs in natural languages, where lots of words with the same stem are not a rare case. Using this input, we have compared three algorithms:

1. "Naive" construction algorithm that appeared in the early works on the AST method. It has the worst-case complexity of $\Theta(n_1^2 + n_2^2 + \cdots + n_m^2)$, where n_i is the length of the i-th string in the collection and m is the collection size;
2. Linear algorithm from Sect. 2, producing the annotated suffix tree data structure in $\Theta(n_1 + n_2 + \cdots + n_m)$ in the worst case;
3. Linear algorithm for enhanced annotated suffix arrays construction from Sect. 3, running in $\Theta(n_1 + n_2 + \cdots + n_m)$ in the worst case.

The results are visualized in Fig. 4. They demonstrate a significant increase in memory efficiency in our new implementation. When compared to the previous suffix tree-based implementation, it uses up to 10 times less memory. Indeed, to construct a usual AST for 100 strings, each of length 500, the software needs 250 MB of memory, while the enhanced implementation requires only 20–25 MB. Such a drastic improvement can be partially explained by the usage of the *numpy* library that wasn't used in the previous implementations (they were based on heavy-weight Python dictionaries to store child nodes), but there is no doubt as well that it is due to the usage of suffix arrays (which in theory should have given us a 2–3x improvement). As for the runtime, our new algorithm shows only a modest improvement over the previous solution.

[3] http://api.yandex.ru/tomita.

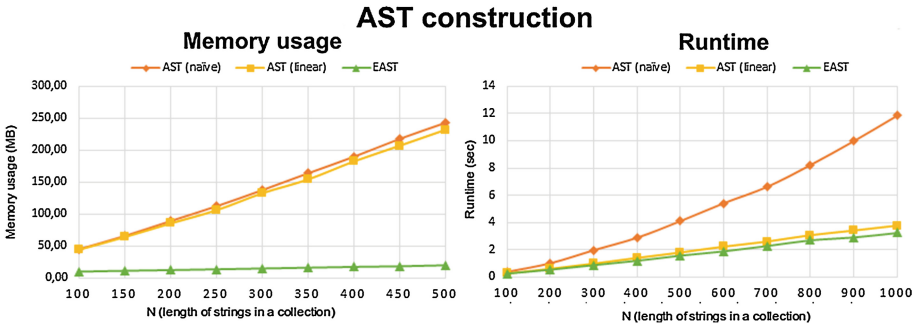

Fig. 4. Experimental results

Our adaptation of a distributional synonym extraction algorithm has shown relatively good results for Russian texts as well. When run over a set of articles from the newspaper "Izvestia" devoted to economics[4], it marked as synonyms, for example, the words *"head"* ("*глава*") and *"CEO"* ("*гендиректор*"), but also wrongly (but rather expectedly) extracted such synonym pairs as *"high"* ("*высокий*") and *"low"* ("*низкий*"). The low precision of this method, however, does not seem to be a big issue: in fact, wrong synonyms usually do not affect the resulting matching score for keyphrases consisting of multiple words (which is chosen as the maximal score among all the possible synonymous phrases).

6 Conclusion

We have developed and implemented a new approach to the construction of annotated suffix trees that uses enhanced suffix arrays as its main underlying data structure. As a result, we have an implementation of the AST method for text analysis which is superior to any other implementation known to us in terms of both memory usage and runtime efficiency.

The corresponding software it the first AST method implementation that has been published as an open-source Python package. It follows the best practices of software engineering, such as self-documented code or unit tests coverage, which makes it easy to integrate and to extend for other researchers and developers.

The EAST package has already been successfully integrated with an automatic Russian text processing system called *LM Monitor*[5] [4]. This system performs the collection and analysis of a Russian Newspaper Web Corpus (*RuNeWC*). The *EAST* package is used in the corpus browser to analyse the relevance of different keyphrases to the newsfeeds. The relevance scores are also used to build the so-called *keyphrase reference graphs* that visualize the relations between keyphrases (computed in a manner similar to *associative rules*). Our enhanced implementa-

[4] http://izvestia.ru/rubric/16.
[5] http://cs.hse.ru/vitext.

tion has proven to work fast and efficiently enough to be able to process large amounts of texts even with modest computational powers.

Acknowledgements. This research carried out in 2015 was supported by "The National Research University 'Higher School of Economics' Academic Fund Program" grant (№ 15-05-0041). The financial support from the Government of the Russian Federation within the framework of the implementation of the 5–100 Programme Roadmap of the National Research University – Higher School of Economics is acknowledged.

References

1. Abouelhoda, M.I., Kurtz, S., Ohlebusch, E.: Replacing suffix trees with enhanced suffix arrays. J. Discrete Algorithms **2**, 53–86 (2004)
2. Barsky, M., Stege, U., Thomo, A.: A survey of practical algorithms for suffix tree construction in external memory. Softw. Pract. Experience **40**(11), 965–988 (2010)
3. Dubov, M., Chernyak, E.: Annotated suffix trees: implementation details. Transactions of Scientific Conference on Analysis of Images, Social Networks and Texts (AIST), pp. 49–57. Springer, Switzerland (2013)
4. Dubov, M., Mirkin, B., Shal, A.: Automatic russian text processing system. Open Systems DBMS **22**(10), 15–17 (2014)
5. Gusfield, D.: Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology. Cambridge University Press, Cambridge (1997)
6. Kinen, J, Sanders, P.: Simple Linear Work Suffix Array Construction. Automata, Languages and Programming. Lecture Notes in Computer Science, pp. 943–2719 (2003)
7. Kasai, T., Lee, G.H., Arimura, H., Arikawa, S., Park, K.: Linear-time longest-common-prefix computation in suffix arrays and its applications. In: Amir, A., Landau, G.M. (eds.) CPM 2001. LNCS, vol. 2089, pp. 181–192. Springer, Heidelberg (2001)
8. Lin, D.: Automatic retrieval and clustering of similar words. In: Proceedings of the 17th International Conference on Computational Linguistics, pp. 768–774 (1998)
9. Manber, U.: Suffix arrays: a new method for on-line string searches. SIAM J. Comput. **22**(5), 935–948 (1993)
10. Mirkin, B., Chernyak, E., Chugunova, O.: Method of annotated suffix tree for scoring the extent of presence of a string in text. Bus. Inf. **3**(21), 31–41 (2012)
11. Pampapathi, R.: Annotated suffix trees for text modelling and classification. Doctoral dissertation, Birkbeck College, University of London, Retrieved from CiteSeerX (2008)
12. Pampapathi, R., Mirkin, B., Levene, M.: A suffix tree approach to anti-spam email filtering. Mach. Learn. **65**(1), 309–338 (2006)
13. Perkins, J.: Python Text Processing with NLTK 2.0 Cookbook. Packt Publishing, Birmingham (2010)
14. Sadakane, K.: Compressed suffix trees with full functionality. Theory of Computing Systems **41**(4), 589–607 (2007)
15. Ukkonen, E.: On-Line Construction of Suffix Trees. Algorithmica **14**(3), 249–260 (1995)
16. Wang, T.: Extracting Synonyms from Dictionary Definitions. Retrieved from Focus on Research, Master dissertation, University of Toronto (2009)